

06-v-model进阶+ref+nextTick

一、v-model进阶

1. 剖析原理

1.1 作用

实现双向数据绑定：

- 数据变了，视图跟着变
- 视图变了，数据跟着变

1.2 原理

v-model本质上是一个语法糖。

例如应用在输入框上，就是 **value** 属性 和 **input** 事件的合写

代码块

```
1 <script setup>
2   import { ref } from 'vue'
3   const msg = ref('')
4 </script>
5 <template>
6   <p>{{ msg }}</p>
7   <input type="text" v-model="msg">
8   <input type="text" :value="msg" @input="msg = $event.target.value" >
9 </template>
```

1.3 注意

\$event 出现在模板中, 用来获取事件的形参

1.4 总结

1. v-model的原理是什么？

答：v-model只是个语法糖，作用在原生输入框上，

本质是 **:value+@input** 的组合

```
1 <input v-model="msg"/>
2
```

```
3 <!-- 等同于 -->
4 <input type="text" :value="msg" @input="msg = $event.target.value" >
```

2. v-model用在组件上

2.1 分析

v-model 除了用在表单元素上, 还可以用在组件上, 实现数据的双向绑定

2.2 需求

实现父组件和子组件数据的双向绑定

2.3 代码演示

BitSelect.vue

```
1 <script setup>
2   // 接收 modelValue
3   const props = defineProps({
4     modelValue: {
5       type: String,
6       default: ''
7     }
8   })
9
10  // 获取 emit
11  const emit = defineEmits()
12 </script>
13
14 <template>
15   <select
16     :value="props.modelValue"
17     @change="emit('update:modelValue', $event.target.value)">
18     <option value="111">北京</option>
19     <option value="222">上海</option>
20     <option value="333">深圳</option>
21     <option value="444">杭州</option>
22     <option value="555">苏州</option>
23   </select>
24 </template>
```

App.vue

```

1  <script setup>
2    import { ref } from 'vue'
3    import BitSelect from './components/BitSelect.vue'
4    const activeId = ref('222')
5  </script>
6
7  <template>
8    <BitSelect v-model="activeId"/>
9  </template>

```

2.4 简化代码

可以使用 `defineModel()` 进行简化上述代码

BitSelect.vue

```

1  <script setup>
2    const model = defineModel()
3  </script>
4
5  <template>
6    <select v-model="model">
7      <option value="111">北京</option>
8      <option value="222">上海</option>
9      <option value="333">深圳</option>
10     <option value="444">杭州</option>
11     <option value="555">苏州</option>
12   </select>
13 </template>

```

2.5 总结

1. 今后在组件上使用 v-model 的规则是？

答：1、父组件绑定 `v-model="数据"`

2、子组件 `const model = defineModel()` 接收数据，子组件可以直接操作 model，即可实现数据双向绑定

二、ref

1. 作用

ref 可以用于 获取原生 DOM 元素 或 组件实例

2. 步骤

1. 声明并绑定 ref
2. 通过 ref.value 获取

3. 案例1: 页面滚动

<https://bitejiuyeye.com/clanguage>

当组件挂载后，让网页滚动到Java课程位置

3.1 静态代码

3.1.1 定义组件

在 **components/bit-card.vue** 中

代码块

```
1  <script setup>
2    const props = defineProps({
3      bgColor: {
4        type: String,
5        default: 'powderblue'
6      }
7    })
8  </script>
9
10 <template>
11   <div class="bit-card">
12     <div
13       class="img"
14       :style="{ background: props.bgColor }"></div>
15     <div class="desc">
16       <h3>内容标题</h3>
17       <p>主讲人: xxx</p>
18       <p class="exercise">立即练习 &gt;&gt;</p>
19     </div>
20   </div>
21 </template>
22
23 <style lang="scss">
24   .bit-card {
25     overflow: hidden;
26     display: flex;
27     flex-direction: column;
28     width: 285px;
29     margin-bottom: 12px;
30     box-shadow: 1px 3px 14px 0 rgba(70, 82, 88, 0.2);
31     border-radius: 6px;
```

```

32     cursor: pointer;
33     .img {
34         height: 204px;
35     }
36     .desc {
37         display: flex;
38         flex-direction: column;
39         padding: 15px 15px 30px 15px;
40         p.exercise {
41             font-size: 14px;
42             text-align: right;
43             color: #2689ff;
44         }
45     }
46 }
47 </style>

```

3.1.2 使用组件

在 App.vue 中

代码块

```

1  <script setup>
2      import BitCard from './components/bit-card.vue'
3  </script>
4
5  <template>
6      <div class="container">
7          <div class="bit-header">
8              <div class="main">头部内容</div>
9          </div>
10         <div class="bit-body">
11             <div class="main">
12                 <div class="bit-c">
13                     <h3>C语言</h3>
14                     <div class="courseList">
15                         <bit-card
16                             bgColor="powderblue"
17                             v-for="n in 8"
18                             :key="n" />
19                     </div>
20                 </div>
21                 <div class="bit-ds">
22                     <h3>数据结构</h3>
23                     <div class="courseList">
24                         <bit-card

```

```
25         bgColor="palevioletred"
26         v-for="n in 8"
27         :key="n" />
28     </div>
29 </div>
30 <div class="bit-java">
31     <h3>Java</h3>
32     <div class="courseList">
33         <bit-card
34             bgColor="purple"
35             v-for="n in 8"
36             :key="n" />
37     </div>
38 </div>
39 </div>
40 </div>
41 <div class="bit-footer">
42     <div class="up"></div>
43     <div class="down">
44         <div class="main">底部内容</div>
45     </div>
46 </div>
47 </div>
48 </template>
49
50 <style>
51     * {
52         margin: 0;
53         padding: 0;
54     }
55 </style>
56 <style lang="scss" scoped>
57     .container {
58         .bit-header {
59             display: flex;
60             justify-content: center;
61             height: 80px;
62             line-height: 80px;
63             background: #fff;
64             box-shadow: 0 3px 6px 0 rgba(7, 35, 68, 0.15);
65             .main {
66                 width: 1200px;
67             }
68         }
69         .bit-body {
70             display: flex;
71             justify-content: center;
```

```

72     background: #f6f8fa;
73     h3 {
74         margin-bottom: 15px;
75     }
76     .main {
77         width: 1200px;
78         .bit-c {
79             padding: 20px 0;
80         }
81         .bit-ds {
82             margin: 40px 0;
83             padding: 20px 0;
84         }
85         .bit-java {
86             padding: 20px 0;
87         }
88         .courseList {

```

3.2 完整代码

scrollIntoView参考文档: <https://developer.mozilla.org/zh-CN/docs/Web/API/Element/scrollIntoView>

代码块

```

1  <script setup>
2      import { onMounted, ref } from 'vue'
3      import BitCard from './components/bit-card.vue'
4
5      const divRef = ref(null)
6
7      // 组件挂载后
8      onMounted(() => {
9          // 滚动到 Java 课程区域
10         divRef.value.scrollIntoView({
11             // 带滚动动画效果
12             behavior: 'smooth'
13         })
14     })
15 </script>
16
17 <template>
18     <div class="bit-java" ref="divRef"></div>
19 </template>

```

4. 案例2: 绘制图表

4.1 安装 echarts 模块

<https://echarts.apache.org/zh/index.html>

代码块

```
1 npm i echarts -S
```

4.2 准备组件

在 **components/bit-chart.vue** 中

代码块

```
1 <script setup></script>
2
3 <template>
4   <div class="bit-chart"></div>
5 </template>
6
7 <style lang="scss" scoped>
8   .bit-chart {
9     width: 800px;
10    height: 500px;
11    margin: 100px auto;
12    border: 3px solid palevioletred;
13  }
14 </style>
```

在 **App.vue** 中

代码块

```
1 <script setup>
2   import BitChart from './components/bit-chart.vue'
3 </script>
4
5 <template>
6   <bit-chart />
7 </template>
```

4.3 完整代码

代码块

```
1 <script setup>
```



```
2   import * as echarts from 'echarts'
3   import { onMounted, ref } from 'vue'
4
5   const divRef = ref(null)
6
7   onMounted(() => {
8     // 基于准备好的dom，初始化echarts实例
9     const myChart = echarts.init(divRef.value)
10    // 绘制图表
11    myChart.setOption({
12      legend: {
13        top: 'bottom'
14      },
15      toolbox: {
16        show: true,
17        feature: {
18          mark: { show: true },
19          dataView: { show: true, readOnly: false },
20          restore: { show: true },
21          saveAsImage: { show: true }
22        }
23      },
24      series: [
25        {
26          name: 'Nightingale Chart',
27          type: 'pie',
28          radius: [50, 250],
29          center: ['50%', '50%'],
30          roseType: 'area',
31          itemStyle: {
32            borderRadius: 8
33          },
34          data: [
35            { value: 40, name: 'rose 1' },
36            { value: 38, name: 'rose 2' },
37            { value: 32, name: 'rose 3' },
38            { value: 30, name: 'rose 4' },
39            { value: 28, name: 'rose 5' },
40            { value: 26, name: 'rose 6' },
41            { value: 22, name: 'rose 7' },
42            { value: 18, name: 'rose 8' }
43          ]
44        }
45      ]
46    })
47  })
48 </script>
```

```
49
50 <template>
51   <div
52     class="bit-chart"
53     ref="divRef"></div>
54 </template>
```

5. 调用组件方法

5.1 静态代码

components/MyForm.vue

```
1 <script setup></script>
2 <template>
3   <div class="login-box">
4     账户: <input type="text" /><br /><br />
5     密码: <input type="password" /><br /><br />
6   </div>
7 </template>
```

App.vue

```
1 <script setup>
2   import MyForm from './components/MyForm.vue'
3 </script>
4
5 <template>
6   <MyForm />
7
8   <button>登 录</button>
9 </template>
10
11 <style>
12   #app {
13     width: 300px;
14     margin: 100px auto;
15   }
16 </style>
```

5.2 完整代码

components/MyForm.vue

```

1  <script setup>
2    // 表单校验
3    const validate = () => {
4      return Math.random() > 0.5 ? true : false
5    }
6
7    // 暴露给组件，目的是父组件可以通过 ref 可以拿到子组件的方法
8    defineExpose({
9      validate
10   })
11 </script>
12 <template>
13   <div class="login-box">
14     账户: <input type="text" /><br /><br />
15     密码: <input type="password" /><br /><br />
16   </div>
17 </template>

```

App.vue

```

1  <script setup>
2    import { ref } from 'vue'
3    import MyForm from './components/MyForm.vue'
4
5    const formRef = ref(null)
6
7    // 登录
8    const onLogin = () => {
9      console.log(formRef.value)
10
11      // 进行校验
12      if (formRef.value.validate()) {
13        console.log('ok')
14      } else {
15        console.log('error')
16      }
17    }
18 </script>
19
20 <template>
21   <MyForm ref="formRef" />
22
23   <button @click="onLogin">登 录</button>
24 </template>

```

```
25
26 <style>
27   #app {
28     width: 300px;
29     margin: 100px auto;
30   }
31 </style>
```

三、nextTick

<https://cn.vuejs.org/api/general.html#nexttick>

1. 作用

等待下一次 DOM 更新刷新的工具方法

代码块

```
1 function nextTick(callback?: () => void): Promise<void>
```

2. 详细解释

当你在 Vue 中更改响应式状态时，最终的 DOM 更新并不是同步生效的，而是由 Vue 将它们缓存在一个队列中，直到下一个“tick”才一起执行。这样是为了确保每个组件无论发生多少状态改变，都仅执行一次更新。

`nextTick()` 可以在状态改变后立即使用，以等待 DOM 更新完成。你可以传递一个回调函数作为参数，或者 `await` 返回的 `Promise`。

3. 代码实现

```
1 <script setup>
2   import { ref, nextTick } from 'vue'
3
4   const count = ref(0)
5
6   const btnRef = ref(null)
7
8   const increment = async () => {
9     // 让 count 自增
10    count.value++
11
12    // 问题：当 count 变了，但获取到的 button 的内容 并不是同步的
13    // 原因：在 Vue 中，数据变了，DOM 的更新是异步的、批量的
```

```
14 // 解决方案：在 nextTick() 时机获取更新后的DOM
15 // nextTick()使用方式有2种：
16 // 1) 一种是借助await+async等待 nextTick() 执行结束
17 // 2) 另一种给nextTick(()=> {}) 传入回调函数，在回调中获取更新后的DOM
18 // 获取 button 的内容
19
20 // 应用场景：数据变了，想获取更新后的DOM，就需要 nextTick() 闪亮登场
21 await nextTick()
22 console.log(btnRef.value.textContent)
23
24 // nextTick(() => {
25 //   console.log(btnRef.value.textContent)
26 // })
27 }
28 </script>
29
30 <template>
31   <button
32     @click="increment"
33     ref="btnRef">
34     {{ count }}
35   </button>
36 </template>
37
38 <style lang="scss" scoped></style>
```